



GUÍA DE PRÁCTICA Nro 07

UNIDAD DIDÁCTICA: Pruebas Unitarias de Software

INDICADOR DE LOGRO DE SESIÓN: Organiza la estructura de una prueba unitaria aplicando el patrón AAA para asegurar la legibilidad y orden del código.

PERIODO ACADEMICO: V **SEMANA:** 7 **FECHA:** 18/05/2026 (D) - 21/05/2026 (N)

DOCENTE: Ing. Erick Moscoso Bravo

I. PROPÓSITO DE LA PRÁCTICA

Aplicar el patrón AAA (Arrange, Act, Assert) y utilizar namespace en pruebas unitaria.

II. CONFIGURACIÓN DEL PROYECTO

Paso 1: Crear la carpeta del proyecto donde deseas

```
cd C:\Users\TuUsuario\Documentos
mkdir semana07
cd semana07
```

Paso 2: Crear estructura de carpetas (src y tests)

Paso 3: Instalar PHPUnit

```
composer require --dev phpunit/phpunit
```

Paso 4: Agregar autoload PSR-04 a composer.json

Abre composer.json y agrega la sección autoload:

```
{
    "autoload": {
        "psr-4": {
            "App\\": "src/"
        }
    },
    "require-dev": {
        "phpunit/phpunit": "^11"
    }
}
```

Paso 5: Actualizar autoload

```
composer dump-autoload
```

III. NAMESPACE

Un namespace organiza clases para evitar conflictos de nombres.

```
namespace App; // Esta clase está en el contenedor "App"
class Calculadora { }
```

Para usar esta clase en otro archivo:

```
use App\Calculadora;
$calc = new Calculadora();
```

IV. PRACTICA EL CODIGO

src/calculadora.php

```
<?php

namespace App;

class Calculadora
{
    public function sumar($a, $b)
    {
        return $a + $b;
    }

    public function restar($a, $b)
    {
        return $a - $b;
    }

    public function multiplicar($a, $b)
    {
        return $a * $b;
    }

    public function dividir($a, $b)
    {
        if ($b == 0) {
            throw new \Exception("División entre cero");
        }
        return $a / $b;
    }
}
```

tests/calculadoraTest.php

```
<?php

namespace Tests;

use PHPUnit\Framework\TestCase;
use App\Calculadora;

class CalculadoraTest extends TestCase
{
    public function testSumar()
    {
        // ARRANGE
        $calc = new Calculadora();

        // ACT
        $resultado = $calc->sumar(5, 3);

        // ASSERT
        $this->assertEquals(8, $resultado);
    }
}
```

```
public function testRestar()
{
    $calc = new Calculadora();
    $resultado = $calc->restar(10, 4);
    $this->assertEquals(6, $resultado);
}

public function testMultiplicar()
{
    $calc = new Calculadora();
    $resultado = $calc->multiplicar(4, 5);
    $this->assertEquals(20, $resultado);
}

public function testDividir()
{
    $calc = new Calculadora();
    $resultado = $calc->dividir(10, 2);
    $this->assertEquals(5, $resultado);
}

public function testDividirEntreCero()
{
    $this->expectException(\Exception::class);
    $calc = new Calculadora();
    $calc->dividir(10, 0);
}
}
```

V. EJECUTAR

```
vendor\bin\phpunit tests
```

Resultado

```
Runtime:      PHP 8.4.21

.....                                             5 / 5 (100%)

Time: 00:00.005, Memory: 16.00 MB

OK (5 tests, 5 assertions)
```

VI. PREGUNTAS

- ¿Qué comando usamos para instalar PHPUnit?
composer require --dev phpunit/phpunit
- ¿Qué significa namespace App;?
Sirve para organizar la clase dentro del grupo App.
- ¿Para qué sirve composer dump-autoload?
Sirve para actualizar la carga automática de clases.
- ¿Qué significa cada letra del patrón AAA?
Arrange: preparar los datos.
Act: ejecutar la acción.
Assert: comprobar el resultado.



VII. ENTREGA

Subir al aula virtual los siguientes archivos ya sea en .zip o .rar

1. src/Calculadora.php
2. tests/CalculadoraTest.php



3. composer.json

(opcional) Si tienes tu repositorio en github envías tu url junto con la practica ya sea en PDF o word

Apellidos y Nombres del Alumno:

Enlace del repositorio: